

Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное образовательное учреждение
высшего образования
«Московский государственный технический университет
имени Н.Э. Баумана
(национальный исследовательский университет)»
(МГТУ им. Н.Э. Баумана)

ФАКУЛЬТЕТ ИНФОРМАТИКА И СИСТЕМЫ УПРАВЛЕНИЯ

КАФЕДРА СИСТЕМЫ ОБРАБОТКИ ИНФОРМАЦИИ И УПРАВЛЕНИЯ

ОТЧЁТ

по рубежному контролю № 1
по курсу «Парадигмы и конструкции языков программирования»
Вариант Д · Предметная область № 27: Преподаватель — Учебный курс

Выполнил: студент Калдас Томас Адел шенуда

Группа: ИУ5И-31Б

Проверил: _____

2026 г.

1. Постановка задачи

Необходимо разработать программу на языке Python, реализующую работу с двумя классами данных, связанными отношениями **«один-ко-многим»** и **«многие-ко-многим»**, и выполняющую три запроса в соответствии с вариантом задания.

Параметр	Значение
Вариант запросов	Д
Номер предметной области	27
Класс 1 (сторона «многие»)	Преподаватель (`Teacher`)
Класс 2 (сторона «один»)	Учебный курс (`Course`)
Связь «многие-ко-многим»	Преподаватели курса (`TeacherCourse`)

Запросы варианта Д

№	Тип связи	Формулировка запроса
Д1	один-ко-многим	Вывести список всех преподавателей, у которых фамилия заканчивается на «ов», и названия их курсов.
Д2	один-ко-многим	Вывести список курсов со средней зарплатой преподавателей на каждом курсе, отсортированный по средней зарплате (с округлением до 2 знаков).
Д3	многие-ко-многим	Вывести список всех курсов, у которых название начинается с буквы «А», и список преподавателей, ведущих эти курсы.

2. Структура данных

Класс	Поля	Назначение
`Teacher` Преподаватель	`id, surname, salary, course_id`	ID, фамилия, зарплата (количественный признак), ID курса (связь 1:M)
`Course` Учебный курс	`id, name`	ID курса, наименование курса
`TeacherCourse` Преподаватели курса	`course_id, teacher_id`	Связующая таблица для отношения M:M

Тестовые данные содержат 5 курсов, 5 преподавателей и 7 записей связующей таблицы. Первичные и вторичные ключи согласованы по идентификаторам. Часть записей связующей таблицы связывает одного преподавателя с несколькими курсами, что и образует отношение «многие-ко-многим».

3. Текст программы

```python
from operator import itemgetter

# ========================================
# 1. Классы данных
# ========================================

class Teacher:
    """Преподаватель"""
    def __init__(self, id, surname, salary, course_id):
        self.id = id
        self.surname = surname
        self.salary = salary      # количественный признак: зарплата
        self.course_id = course_id  # связь «один-ко-многим»

class Course:
    """Учебный курс"""
    def __init__(self, id, name):
        self.id = id
        self.name = name

class TeacherCourse:
    """'Преподаватели курса' для реализации связи «многие-ко-многим»"""
    def __init__(self, course_id, teacher_id):
        self.course_id = course_id
        self.teacher_id = teacher_id

# ========================================
# 2. Тестовые данные
# ========================================

# Учебные курсы
courses = [
    Course(1, 'Алгебра'),
    Course(2, 'Астрономия'),
    Course(3, 'История'),
    Course(4, 'Физика'),
    Course(5, 'Английский язык'),
]

# Преподаватели
teachers = [
    Teacher(1, 'Иванов', 50000, 1),
    Teacher(2, 'Петров', 60000, 2),
    Teacher(3, 'Сидоров', 45000, 3),
    Teacher(4, 'Кузнецов', 70000, 4),
    Teacher(5, 'Смирнов', 55000, 5),
]

# Связь «многие-ко-многим»
teachers_courses = [
    TeacherCourse(1, 1),
    TeacherCourse(2, 2),
    TeacherCourse(3, 3),
    TeacherCourse(4, 4),
    TeacherCourse(5, 5),
    # дополнительные связи (один преподаватель ведёт несколько курсов)
    TeacherCourse(1, 2),
    TeacherCourse(3, 5),
]

def main():
    """Основная функция"""

    # Соединение данных «один-ко-многим»
    one_to_many = [(t.surname, t.salary, c.name)
                   for c in courses
```

```python
                    for t in teachers
                    if t.course_id == c.id]

    # Соединение данных «многие-ко-многим»
    many_to_many_temp = [(c.name, tc.course_id, tc.teacher_id)
                         for c in courses
                         for tc in teachers_courses
                         if c.id == tc.course_id]

    many_to_many = [(t.surname, t.salary, course_name)
                    for course_name, course_id, teacher_id in many_to_many_temp
                    for t in teachers if t.id == teacher_id]

    # ========================================
    # 3. Запросы (вариант Д)
    # ========================================

    print('--- Задание Д1 ---')
    # Д1: преподаватели, у которых фамилия заканчивается на «ов», и их курсы
    res_1 = [(t.surname, c.name) for t in teachers for c in courses
             if t.course_id == c.id and t.surname.endswith('ов')]
    print(res_1)

    print('\n--- Задание Д2 ---')
    # Д2: курсы со средней зарплатой преподавателей, сортировка по средней зарплате
    res_2_unsorted = []
    for c in courses:
        # Преподаватели данного курса
        c_teachers = list(filter(lambda i: i[2] == c.name, one_to_many))
        if len(c_teachers) > 0:
            # Зарплаты преподавателей курса
            c_salaries = [sal for _, sal, _ in c_teachers]
            # Средняя зарплата = сумма / количество
            avg_salary = sum(c_salaries) / len(c_salaries)
            res_2_unsorted.append((c.name, round(avg_salary, 2)))

    res_2 = sorted(res_2_unsorted, key=itemgetter(1), reverse=True)
    print(res_2)

    print('\n--- Задание Д3 ---')
    # Д3: курсы, название которых начинается с буквы «А», и их преподаватели
    res_3 = {}
    for c in courses:
        if c.name.startswith('A'):
            c_teachers = list(filter(lambda i: i[2] == c.name, many_to_many))
            teacher_names = [x for x, _, _ in c_teachers]
            res_3[c.name] = teacher_names

    print(res_3)

if __name__ == '__main__':
    main()

```

4. Результаты выполнения

Программа запущена командой `python3 main.py`. Результат работы:

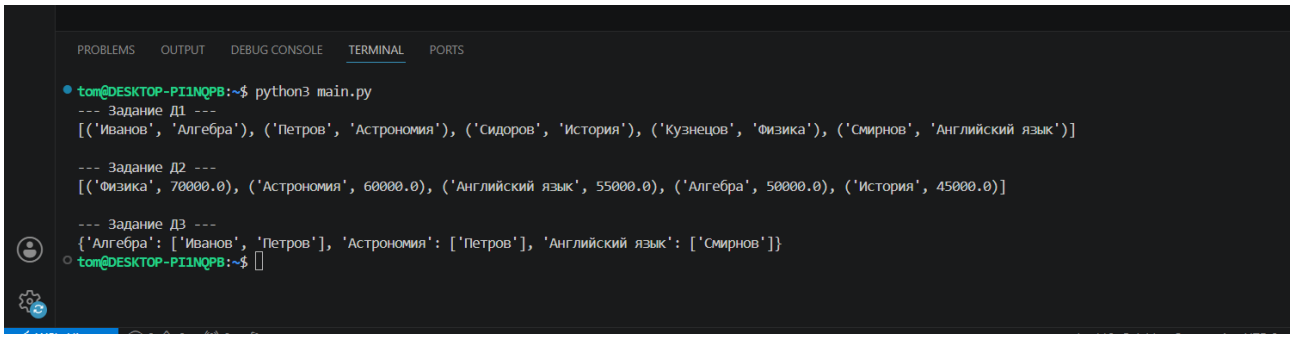


Рисунок 1 — Результат выполнения программы в терминале

Разбор результатов

Запрос	Результат
Д1	Все 5 фамилий (Иванов, Петров, Сидоров, Кузнецов, Смирнов) заканчиваются на «ов», поэтому в результат вошли все преподаватели вместе с названиями курсов.
Д2	Курсы отсортированы по убыванию средней зарплаты: Физика (70000.0) → Астрономия (60000.0) → Английский язык (55000.0) → Алгебра (50000.0) → История (45000.0).
Д3	Курсов на «А» три: Алгебра (Иванов, Петров), Астрономия (Петров), Английский язык (Смирнов). Алгебру ведут двое — за счёт связи «многие-ко-многим».

5. Вывод

В ходе работы были реализованы классы данных «Преподаватель», «Учебный курс» и «Преподаватели курса», связанные отношениями «один-ко-многим» и «многие-ко-многим». Запросы Д1–Д3 выполнены с использованием списковых включений (list comprehensions), словарей, функций высшего порядка (`filter`, `sorted`, `lambda`) и встроенных функций `sum`, `len`, `round`. Результаты соответствуют ожидаемым.

6. Репозиторий GitHub

Отчёт и исходный код проекта размещены в репозитории на GitHub:

https://github.com/thomas3del/Report_RK1_Variant_D